Report - Version 9: Transformer Block

Introduction

Version 9 extends the explicit scaled dot-product attention from Version 8 into a complete single Transformer block.

The same 440-character English corpus, 27-character vocabulary, four-character context, trainable character embeddings, positional embeddings, reproducible train/validation split, mini-batch optimization and autoregressive generation are preserved. The main change is architectural: training now predicts the next character at every context position and the model adds causal self-attention, residual connections, layer normalization and a position-wise feed-forward network.

```
Characters
  |
Integer identifiers
  |
4-character context + 4 sequential targets
  |
Character + positional embeddings
  |
Causal self-attention
  |
Residual + LayerNorm
  |
Feed-forward network
  |
Residual + LayerNorm
  |
Vocabulary logits at every position
  |
Generated text
```

Goal of Version 9

- reuse the same corpus, vocabulary, four-character context and 8-dimensional embeddings from Version 8;
- change the target from one next character to four sequential next-character targets, for example mode -> odel;
- apply a causal mask so each position can attend only to itself and earlier positions;
- add attention output projection, residual connections, LayerNorm and a feed-forward sublayer;
- train all positions while using only the final-position logits during autoregressive generation;
- verify shapes, parameter count, causal masking, gradients, updates, loss histories, reproducibility and generation.

Architecture

The complete Version 9 flow is:

```
Corpus: 440 characters
  |
Vocabulary: 27 characters
  |
436 sequential context-target examples
  |
Input tensor:  (436, 4)
Target tensor: (436, 4)
  |
Character embeddings: (27, 8) + positions: (4, 8)
  |
Queries / keys / values: 4 x 16 per example
  |
Causal attention matrix: 4 x 4 per example
  |
Attention output projection: 16 -> 8
  |
Residual + LayerNorm
  |
Feed-forward network: 8 -> 16 -> 8
  |
Residual + LayerNorm
  |
Vocabulary logits: (4, 27) per example
  |
Reproducible 80 / 20 split + mini-batch updates
  |
Sliding-window autoregressive generator
```

Sequential context targets

Each example still contains four input characters, but Version 9 learns four aligned next-character predictions. With input mode, the target is odel. Position 0 predicts o from m; position 1 predicts d from mo; position 2 predicts e from mod; and position 3 predicts l from mode. The causal mask makes these training targets consistent with autoregressive generation.

Context windows, split and mini-batches

```
training examples = 348
validation examples = 88
batch size = 32
updates per epoch = 11
```

Only training examples are shuffled and updated. Validation measures loss without changing the parameters. Neighboring windows overlap, so this split remains a lightweight learning monitor rather than a strong estimate of generalization to unseen text.

Trainable parameters

The model contains 13 trainable TensorFlow variables and exactly 1264 trainable parameters:

```
Character embeddings        27 x 8   = 216
Positional embeddings         4 x 8   =  32
Query projection              8 x 16  = 128
Key projection                8 x 16  = 128
Value projection              8 x 16  = 128
Attention output projection  16 x 8   = 128
LayerNorm 1 scale + shift       8 + 8 =  16
Feed-forward input            8 x 16  = 128
Feed-forward output          16 x 8   = 128
LayerNorm 2 scale + shift       8 + 8 =  16
Output weights                8 x 27  = 216
-------------------------------------------
Total                                 = 1264
```

Transformer block details

Character IDs select trainable 8-dimensional embeddings. A trainable positional vector is added to each of the four positions. The resulting sequence is projected to 16-dimensional queries, keys and values.

```
Q = X Wq
K = X Wk
V = X Wv

scores = Q K^T / sqrt(dk)
masked_scores = causal_mask(scores)
A = softmax(masked_scores)
H = A V
```

The attention output is projected from 16 back to 8 values per position so it can be added to the positioned input through the first residual connection. Layer normalization follows. A position-wise feed-forward network expands 8 values to 16, applies the non-linearity, projects back to 8, then a second residual connection and LayerNorm complete the block. No bias terms are used in attention, feed-forward or vocabulary projections; both LayerNorm operations use trainable scale and shift.

Causal mask

```
[[1, 0, 0, 0],
 [1, 1, 0, 0],
 [1, 1, 1, 0],
 [1, 1, 1, 1]]
```

Loss, training and reproducibility

Sparse softmax cross-entropy is computed at all four sequence positions. The per-position losses have shape (batch, 4) and their mean is the scalar objective. tf.GradientTape calculates gradients for all 13 trainable variables, and manual mini-batch gradient descent applies the updates.

```
Learning rate: 1.0
Batch size:     32
Epochs:         100
Seed:           42
```

Saved training result

```
Initial train loss:       3.298020
Final train loss:         1.915883
Initial validation loss:  3.299714
Final validation loss:    2.208963
Best validation loss:     2.130791
Best validation epoch:    67
```

Training loss decreases substantially. Validation improves early, reaches its best value at epoch 67, and later fluctuates while training continues to improve, which is consistent with mild overfitting on this very small dataset. The reported values are those saved in the current notebook execution; small numerical differences can appear across TensorFlow environments.

Inspection of learned probabilities

For context mode, the highest final-position probabilities in the saved run are:

```
space  0.3648
r      0.1843
d      0.0762
l      0.0639
t      0.0583
m      0.0465
```

Inspection of causal attention

```
        m       o       d       e
m   1.0000  0.0000  0.0000  0.0000
o   0.9234  0.0766  0.0000  0.0000
d   0.2832  0.3961  0.3207  0.0000
e   0.1037  0.4917  0.2196  0.1849
```

Every row sums to approximately 1 and all entries above the main diagonal are zero, confirming the causal constraint.

Autoregressive generation

Generation still uses a sliding four-character window. At every step the entire Transformer block is recomputed for the current context. Only the logits from the final sequence position are converted to probabilities and sampled, then the oldest context character is discarded and the new character is appended.

With seed 42, five independent samples from context mode in the saved execution are:

```
r, d, s, r, space
```

The 300-character demonstration begins:

```
moderetr win mawiom n.
sinttl macors mod manes lexper fconged clerning lamamp belers,
th comakerincte crsmay rin tleleaserks pamake codvame win mexpl...
```

Included checks

- the input and target tensors both have shape (436, 4);
- all 13 trainable variables have the expected shapes and total exactly 1264 parameters;
- the forward pass produces embeddings (1, 4, 8), Q/K/V (1, 4, 16), attention (1, 4, 4), Transformer output (1, 4, 8) and logits (1, 4, 27);
- the causal mask is lower triangular and future attention weights are exactly zero;
- attention rows and next-character probability distributions sum to 1;
- all gradients are present and finite, and every trainable variable changes during training;
- training and validation indices are disjoint and loss histories remain finite;
- retraining and autoregressive generation are reproducible with the same seed.

The notebook finishes its test section with All checks passed.

What Version 9 teaches

Version 9 is the first stage in the project that has the internal structure of a Transformer block rather than only an attention mechanism. The key conceptual progression is:

```
context -> embeddings + positions
        -> Q / K / V
        -> causal attention
        -> output projection + residual + LayerNorm
        -> feed-forward + residual + LayerNorm
        -> logits at every position
        -> loss over all positions
```

The important distinction from Version 8 is not simply adding more code. The new code introduces architectural roles that later decoder-only Transformers rely on: causal information flow, residual paths, normalization and position-wise non-linear processing.

Limitations

- the context length remains fixed at four characters and the corpus contains only 440 characters;
- the model uses one attention mechanism rather than multi-head attention;
- only one Transformer block is used; there is no stacking, dropout or learning-rate schedule;
- overlapping windows appear across the random train/validation split, so validation is not a robust generalization benchmark;
- generation retains the final epoch parameters rather than automatically restoring the best validation checkpoint;
- the implementation is deliberately explicit, so parameter passing is becoming verbose and is a natural candidate for refactoring in a later version.

Conclusion

Version 9 successfully turns the Version 8 attention model into a genuine single causal Transformer block. It preserves the small, inspectable character-level setting while introducing the structural components needed for the next step toward a more complete decoder-only language model.

At this point the extra complexity is still didactically justified: each added operation corresponds to a distinct Transformer concept. A later version can move these parameters into a class or module without hiding the mathematics already made explicit here.